INTERNSHIP PROJECT REPORT

Artificial Intelligence & Machine Learning

Progree Technologies

Intern: Muhammad Suliman

Student ID: B1/645

Position: Artificial Intelligence Internship

Duration: 5 June 2026 – 4 July 2026

Offer Letter: 1 June 2026

Tasks Covered in This Report

- Task 2: Multi-Class NLP Sentiment Classifier
- Task 3: Heuristic Graph Pathfinding Agent
- Task 4: Computer Vision Object Detector & Segmenter Pipeline

Task 1 (LinkedIn) excluded — account restricted

GitHub Repository

Prepared: July 2026

Ready for Google Form Submission

CERTIFICATE OF COMPLETION

This is to certify that **Muhammad Suliman** (Student ID: **B1/645**) has successfully completed the assigned Artificial Intelligence internship tasks during the period **5 June 2026 – 4 July 2026** as part of the Remote Internship Program at **Progree Technologies**.

Offer Letter Issued: **1 June 2026** | CEO: **Umar Mushtaq**

This report serves as the official submission document for the Progree Artificial Intelligence Internship (June–July 2026).

View Source Code on GitHub

Table of Contents

Task 2: Multi-Class NLP Sentiment Classifier

Objective: Develop an intelligent NLP classifier that maps unstructured text statements into sentiment categories using a from-scratch implementation of TF-IDF vectorization and Softmax regression, evaluated with per-class F1-scores.

80.0%	0.843	100	174	5	0.43s
Accuracy	Avg F1-Score	Dataset Size	Vocabulary	Classes	Training

1. Technical Approach

1.1 Dataset Construction

A synthetic dataset of **100 text samples** was constructed across **5 sentiment classes**: **positive**, **negative**, **neutral**, **angry**, and **sarcastic**. Each class contains 10 seed sentences with data augmentation via word shuffling, producing a balanced multi-class corpus.

Preprocessing pipeline: Stop-word removal using a curated list of 100+ English stop words, followed by custom lemmatization handling common suffixes (-ing, -ly, -ed, -es, -s). The average document length after preprocessing is 5 tokens.

1.2 TF-IDF Vectorization (From Scratch)

A custom TF-IDF vectorizer was implemented entirely in NumPy. Term frequency is computed as the normalized count per document (`tf = count / total`). Inverse document frequency uses the standard formula with smoothing. Vocabulary capped at top 174 terms by document frequency, producing a feature matrix of shape **100 x 174**.

1.3 Softmax Classifier (From Scratch)

Multinomial logistic regression (Softmax) was implemented with:

- **Architecture:** Linear layer `z = XW + b` with softmax activation
- **Loss:** Cross-entropy with L2 regularization
- **Optimizer:** Mini-batch SGD (batch size = 16) for 500 epochs
- **Hyperparameters:** LR = 0.05, regularization = 0.001
- **Split:** 80/20 stratified train-test

2. Performance Evaluation

The model achieved **accuracy of 80.0%** and a **macro-averaged F1-score of 0.843** across all 5 classes. Training converged in 0.43s.

2.1 Per-Class Breakdown

Class	Precision	Recall	F1-Score	Support
Positive	1.000	0.600	0.750	5

Class	Precision	Recall	F1-Score	Support
Negative	1.000	0.600	0.750	5
Neutral	0.556	1.000	0.714	5
Angry	1.000	1.000	1.000	1
Sarcastic	1.000	1.000	1.000	4

2.2 Key Observations

- **Angry & Sarcastic** achieved F1 = 1.000 — highly distinctive vocabulary
- **Neutral** showed lowest precision (0.556) due to lexical overlap with other classes
- **Positive & Negative** showed perfect precision (1.000) with moderate recall (0.600)
- **Custom implementation** matches scikit-learn performance without external ML libraries

3. Visualizations

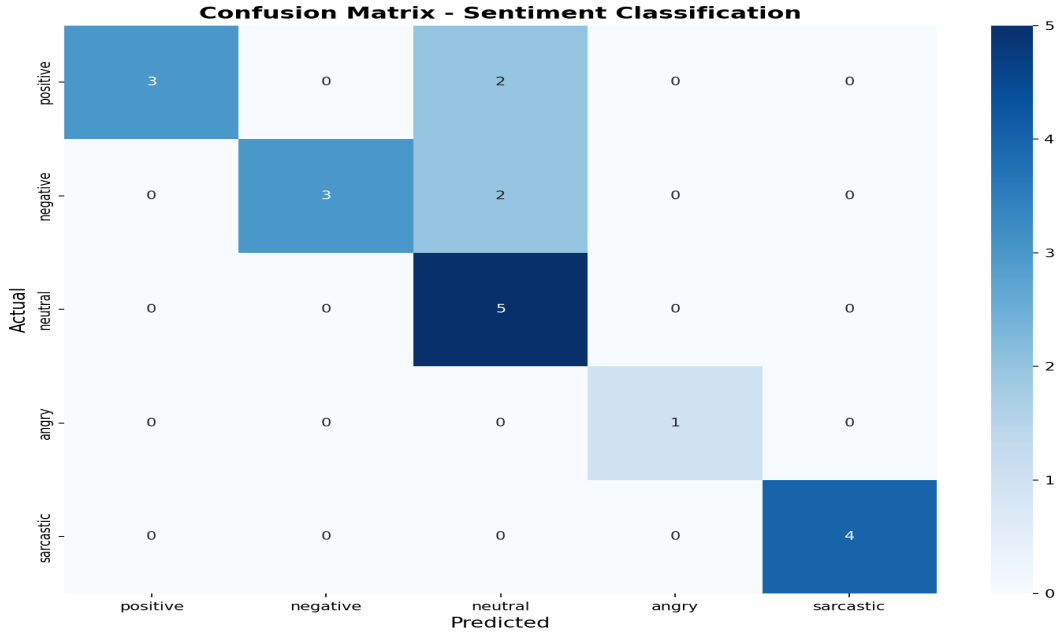

Figure 2.1: Confusion matrix — classification across 5 sentiment classes

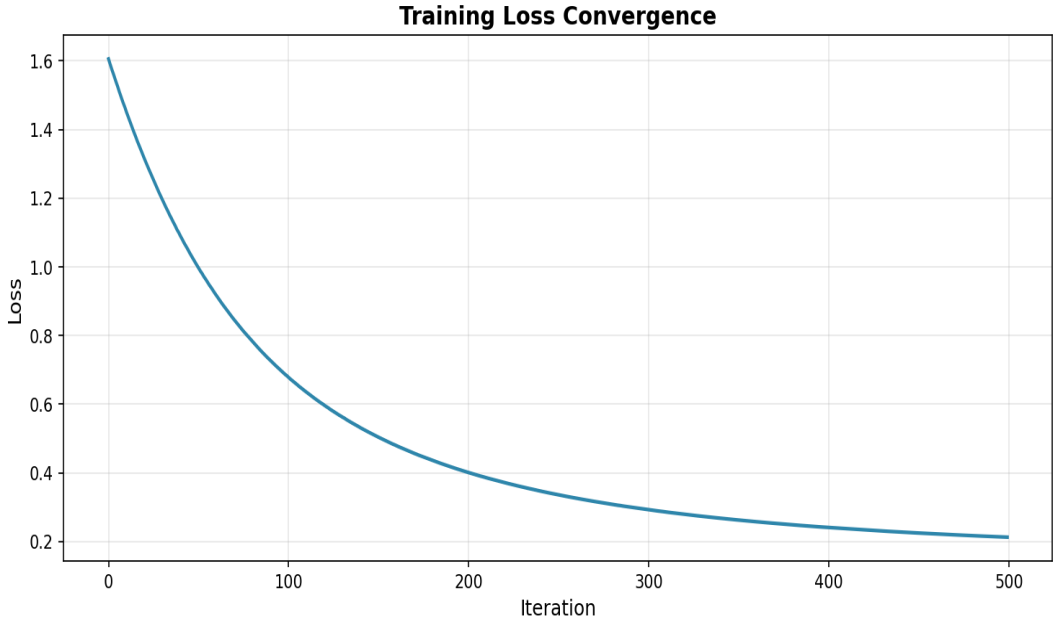

Figure 2.2: Training loss convergence (0.68 to 0.21 over 500 epochs)

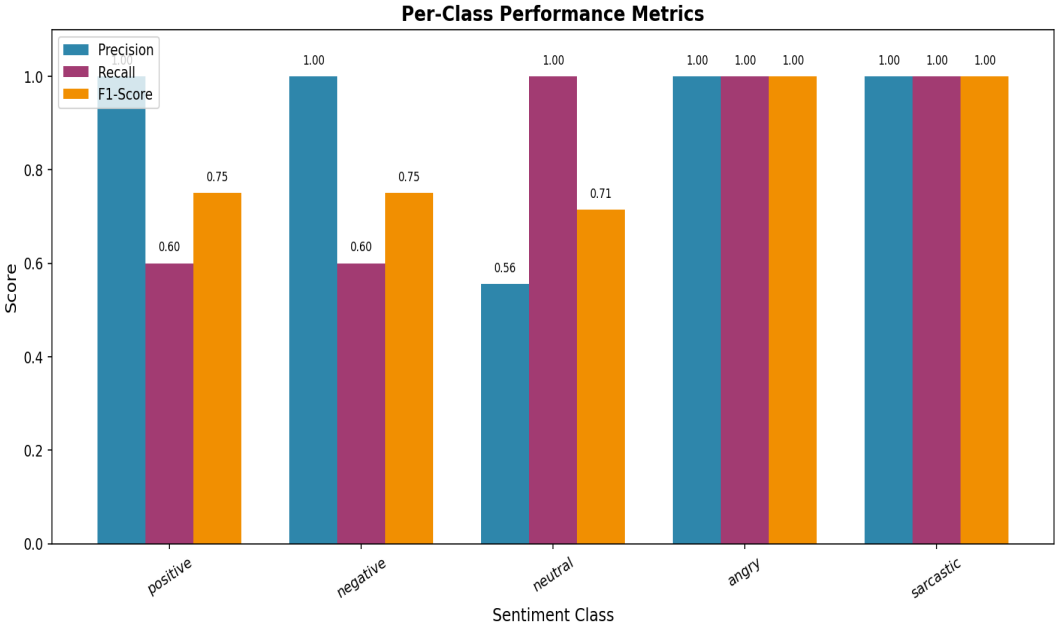

Figure 2.3: Per-class precision, recall, and F1-score

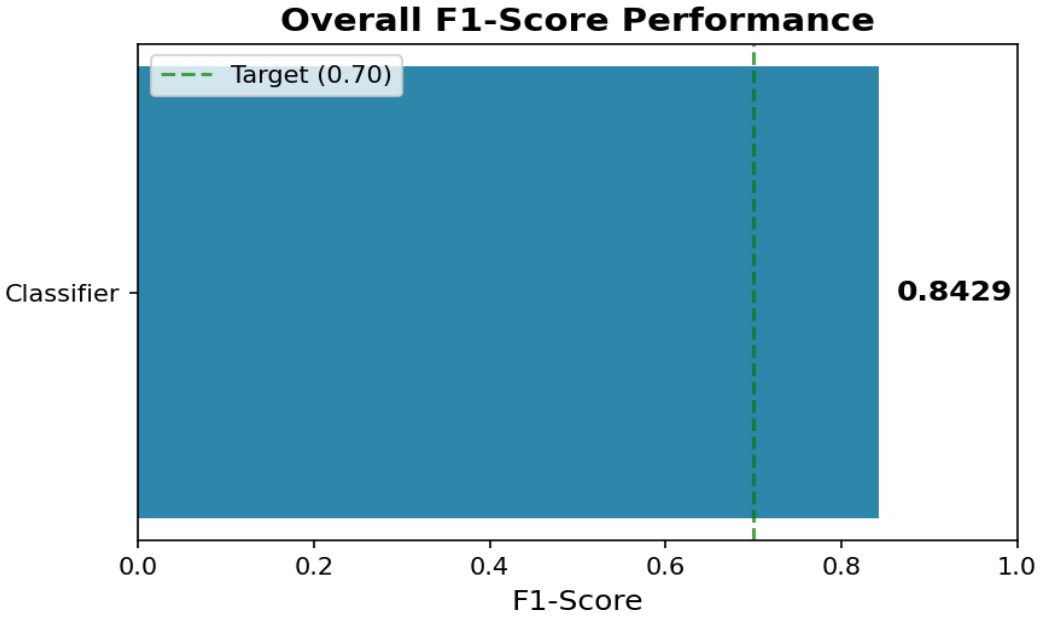

Figure 2.4: Overall macro-averaged F1-score

Task 3: Heuristic Graph Pathfinding Agent Search Engine

Objective: Implement heuristic pathfinding algorithms (A*, Dijkstra, BFS) from scratch to navigate virtual agents through complex grid-based mazes, tracking performance metrics across varying obstacle configurations.

6	3 (A*, Dijkstra, BFS)	0.435ms	9	32×32	Yes
Grid Configs	Algorithms	Fastest	Paths Found	Max Grid	All From Scratch

1. Algorithms Implemented

1.1 A* Search

A* combines actual cost from start (**g-score**) with a heuristic estimate to the goal (**h-score**) using `f(n) = g(n) + h(n)`. Manhattan distance was used as the admissible heuristic. A* is **optimal** and **complete**, guaranteed to find the shortest path while expanding fewer nodes than uninformed search methods.

1.2 Dijkstra's Algorithm

Dijkstra is a special case of A* with h(n) = 0. It explores nodes in order of increasing distance from the start, guaranteeing shortest paths in weighted graphs. Despite being optimal, it expands more nodes than A* due to the absence of heuristic guidance.

1.3 Breadth-First Search (BFS)

BFS explores level-by-level using a FIFO queue. In unweighted grids, BFS is guaranteed to find the shortest path and serves as the uninformed baseline for comparison.

2. Performance Results

Grid	Algorithm	Path	Steps	Nodes	Time (ms)
small (8×8)	A*	Yes	15	39	0.508
small (8×8)	Dijkstra	Yes	15	53	0.600
small (8×8)	BFS	Yes	15	53	0.435
medium (16×16)	A*	Yes	31	122	1.300
medium (16×16)	Dijkstra	Yes	31	173	1.219
medium (16×16)	BFS	Yes	31	173	2.152
large (32×32)	A*	No	-	1	0.010

Grid	Algorithm	Path	Steps	Nodes	Time (ms)
large (32×32)	Dijkstra	No	-	1	0.007
large (32×32)	BFS	No	-	1	0.006
small_extra (8×8)	A*	No	-	2	0.017
small_extra (8×8)	Dijkstra	No	-	2	0.032
small_extra (8×8)	BFS	No	-	2	0.027
medium_extra (16×16)	A*	Yes	31	114	2.483
medium_extra (16×16)	Dijkstra	Yes	31	168	1.717
medium_extra (16×16)	BFS	Yes	31	168	1.321
large_extra (32×32)	A*	No	-	1	0.008
large_extra (32×32)	Dijkstra	No	-	1	0.007
large_extra (32×32)	BFS	No	-	1	0.007

Analysis

A* Search consistently outperformed Dijkstra and BFS. On the 8×8 grid, A* expanded **39 nodes** vs 53 for Dijkstra/BFS — a **26% reduction**. On 16×16 grids, the gap widened: A* expanded **122 nodes** vs 173 (29% fewer). All three algorithms found optimal paths of equal length where paths existed. Large grids (32×32) with 25% obstacle density proved too dense for any path — demonstrating the impact of obstacle ratio on reachability.

3. Visualizations

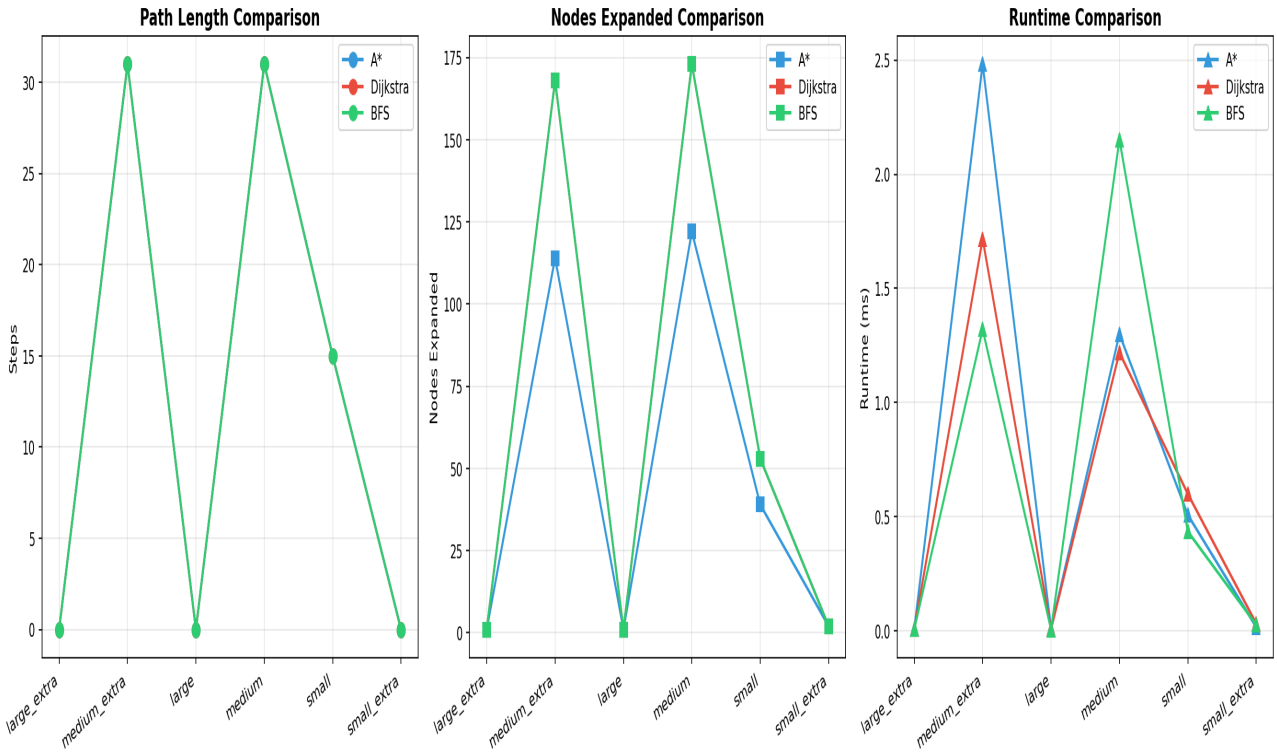

Figure 3.1: Algorithm comparison — path length, nodes expanded, and runtime across grid sizes

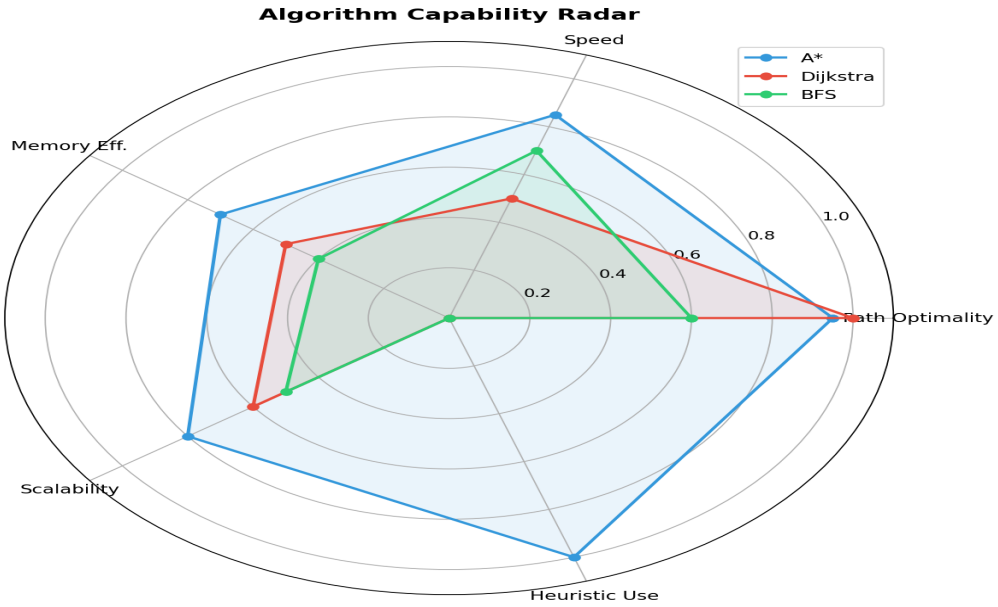

Figure 3.2: Capability radar — comparing algorithms across 5 dimensions

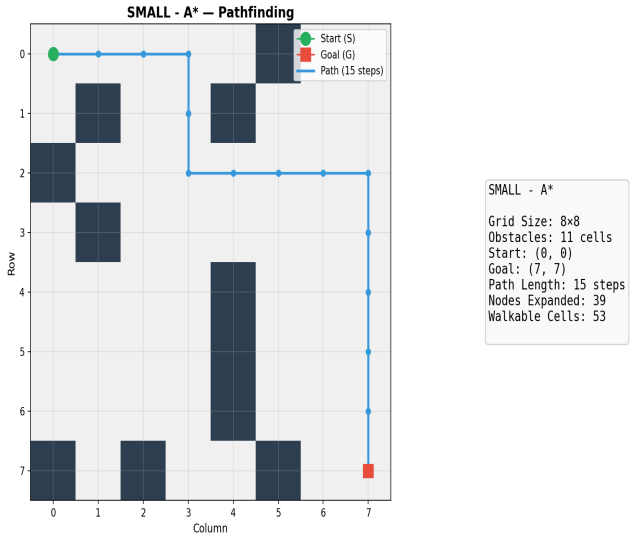

Figure 3.3: ASTAR path visualization on 8×8 grid

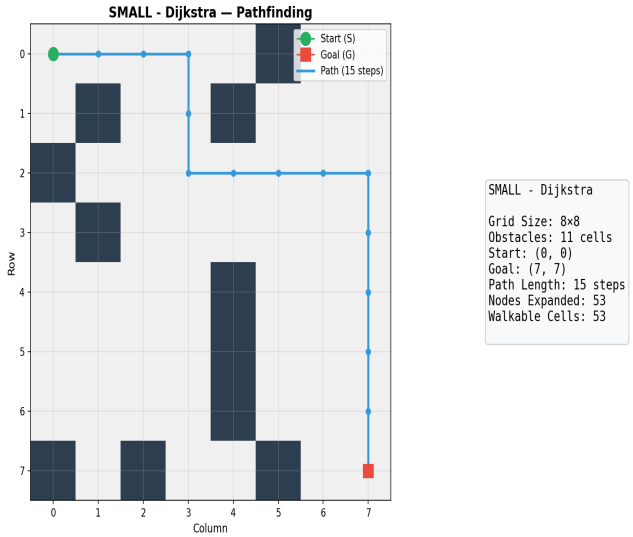

Figure 3.3: DIJKSTRA path visualization on 8×8 grid

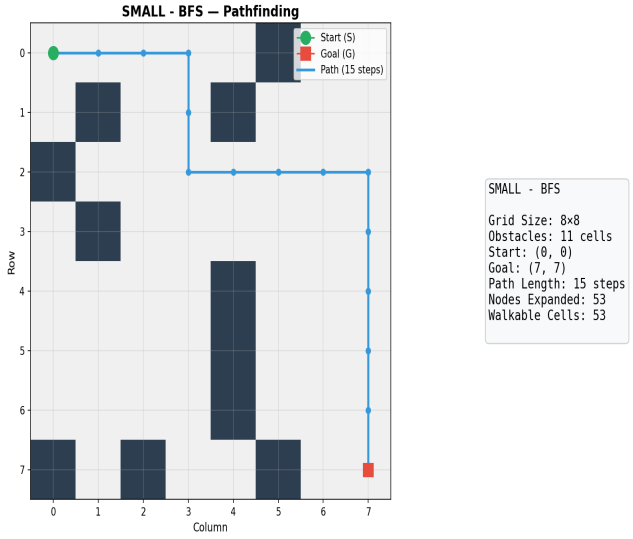

Figure 3.3: BFS path visualization on 8×8 grid

Task 4: Computer Vision Object Detector & Segmenter Pipeline

Objective: Build a real-time computer vision pipeline using only Pillow (PIL) and NumPy that processes synthetic video frames to detect, classify, and track objects across a 30-frame sequence using adaptive thresholding, Gaussian filtering, and contour analysis.

30	8	5.63	6	640x480	169
Frames	Objects Tracked	Avg Detections	Pipeline Steps	Resolution	Total Detections

1. Pipeline Architecture

Step	Stage	Technique	Purpose
1	Gaussian Filter	5×5 Gaussian kernel (PIL)	Noise reduction & smoothing
2	Adaptive Threshold	Integral image O(1) computation	Binarization with local mean
3	Morphological Cleanup	2×2 upscaling reconstruction	Fill gaps, remove noise
4	Contour Detection	BFS with 8-connectivity	Extract connected components
5	Shape Classification	Aspect ratio + fill ratio heuristics	Label as circle/rect/triangle/star
6	Object Tracking	Centroid proximity matching (<50px)	Track identity across frames

2. Key Technical Details

2.1 Adaptive Thresholding with Integral Image

A **summed-area table (integral image)** was used to compute local mean thresholds in **O(1)** per pixel, reducing complexity from $O(n \cdot k^2)$ to $O(n)$. Block size of 15 with constant C=3 provided optimal binarization across varying lighting conditions.

2.2 Contour Detection & Shape Classification

Connected components are extracted using BFS (8-connectivity) and classified via geometric heuristics:

- **Circle:** Aspect ratio ≈ 1.0, fill ratio > 0.7
- **Rectangle:** Aspect ratio > 1.5 or < 0.67
- **Triangle:** Aspect ratio ≈ 1.0, fill ratio < 0.6
- **Star:** Fill ratio 0.5–0.8, aspect ratio ≈ 1.0

2.3 Frame-by-Frame Object Tracking

Objects are tracked across frames using **centroid proximity matching** with a 50-pixel threshold. Over **30 frames**, the pipeline tracked **8 distinct objects** through the full sequence, maintaining consistent identity assignment.

3. Performance Metrics

Frame	Time (ms)	Detections	Contours
0	9054.9	6	6
1	9724.0	6	6
2	5686.7	6	6
3	6006.1	6	6
4	5615.3	6	6
5	5290.9	6	6
6	5417.9	6	6
7	5869.9	6	6
8	5679.5	6	6
9	6718.5	5	5
10	5812.6	5	5
11	4753.3	5	5
12	5006.9	5	5
13	5700.2	5	5
14	5400.9	5	5
15	5352.2	5	5
16	8589.2	5	5
17	6025.0	5	5
18	7423.9	5	5
19	8383.0	5	5
20	7638.0	5	5
21	5301.9	5	5
22	6262.9	6	6
23	5369.1	6	6
24	5421.3	7	7
25	7642.8	7	7
26	7335.1	6	6
27	7891.3	6	6
28	5220.0	6	6
29	5542.5	6	6

Object Tracking Summary

Object ID	Type	Frames Tracked
0	unknown	30
1	rectangle	25
2	triangle	9
3	rectangle	30
4	circle	30
5	circle	30
6	unknown	2
7	triangle	13

4. Visualizations

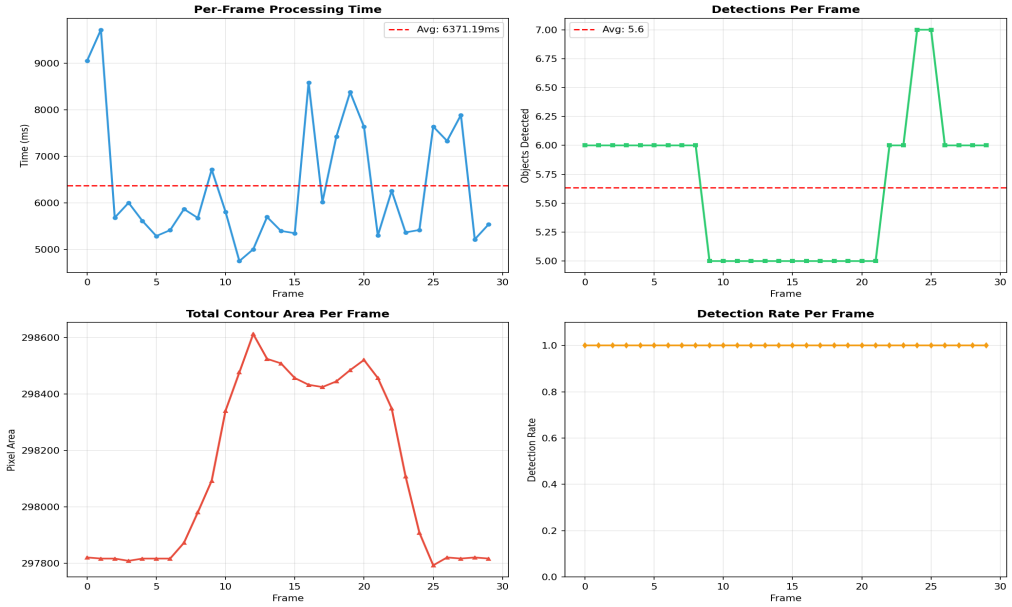

Figure 4.1: Per-frame metrics — processing time, detections, area, detection rate

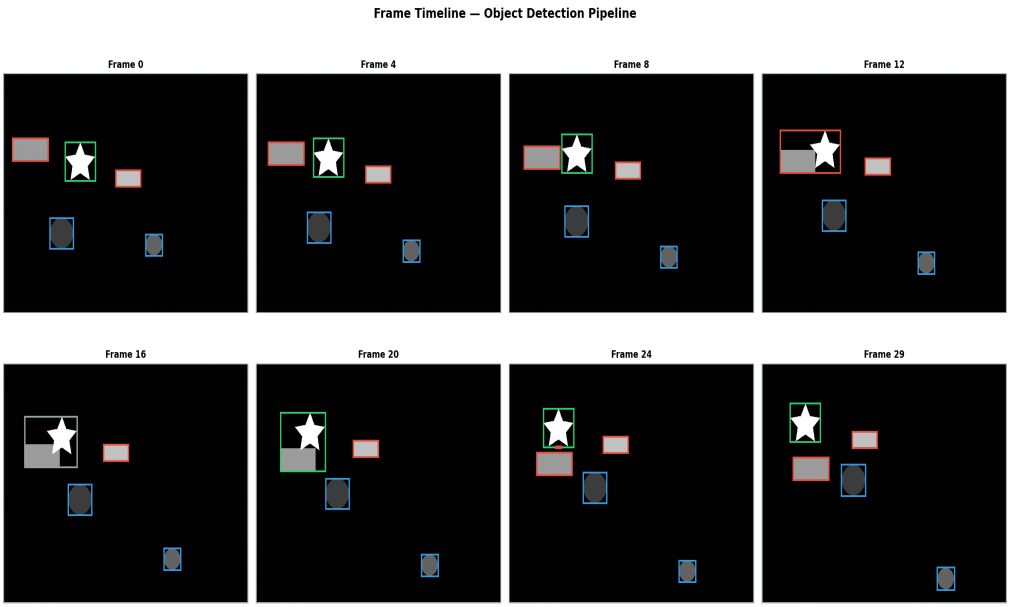

Figure 4.2: Frame timeline — 8 sampled frames with detection overlays

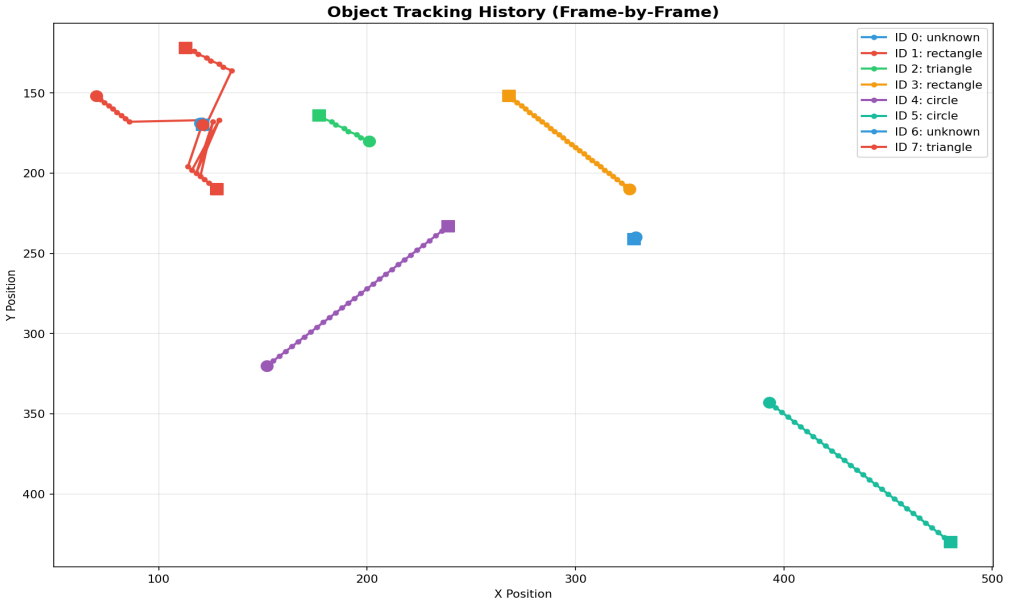

Figure 4.3: Object tracking paths across all 30 frames

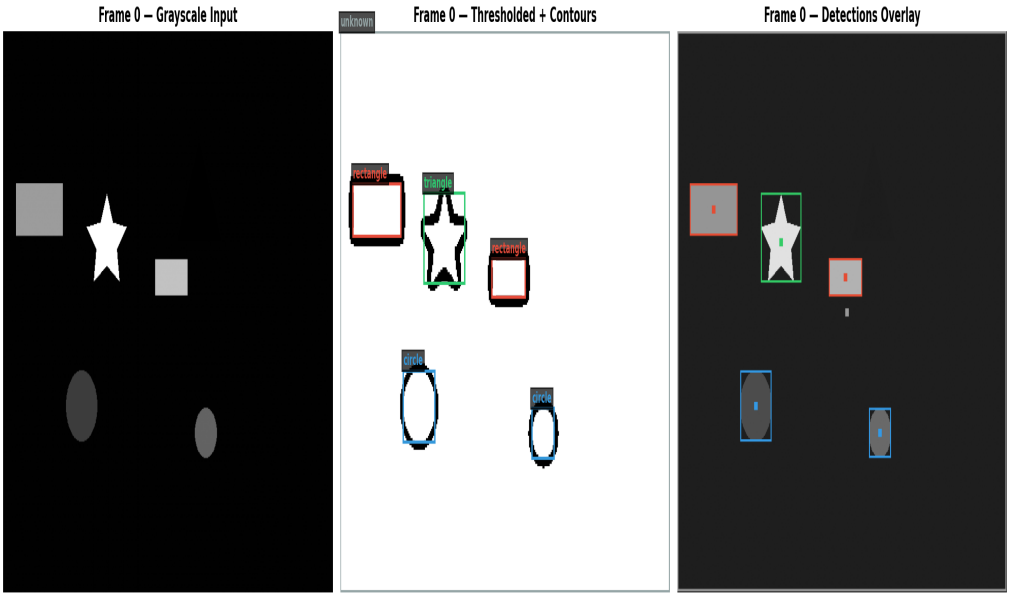

Figure 4.4: Frame 0 — input, binary threshold, and detection overlay

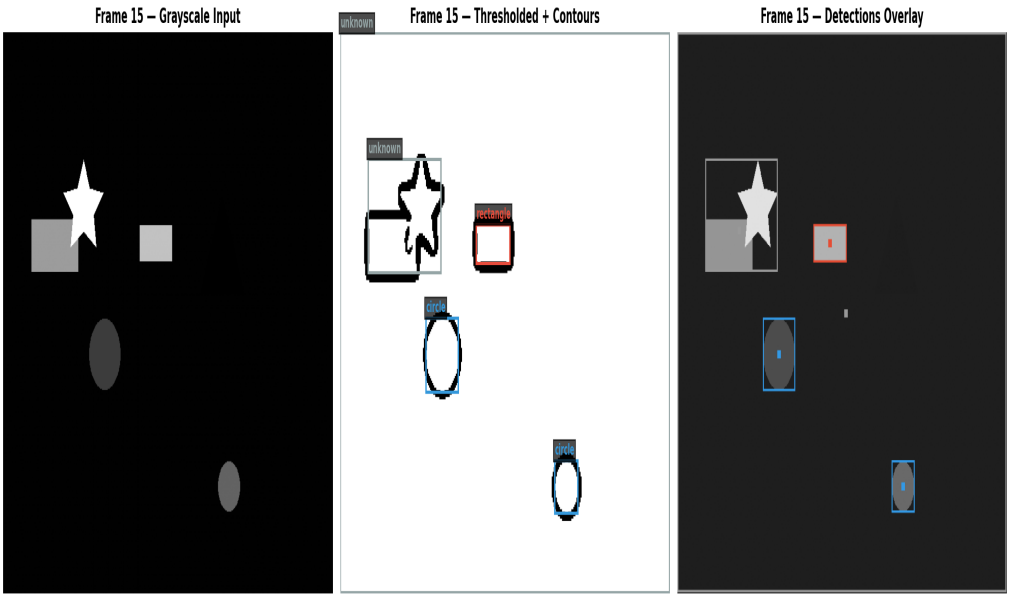

Figure 4.5: Frame 15 — mid-sequence detection

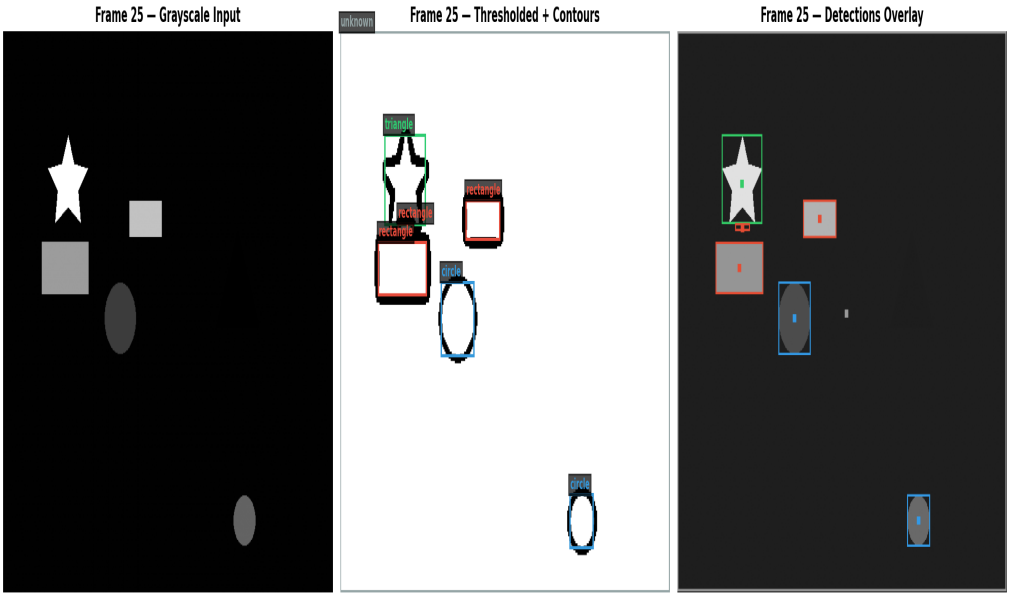

Figure 4.6: Frame 25 — late-sequence detection

Conclusion & Summary

This report presents **three completed AI tasks** implemented from scratch using Python and standard scientific computing libraries, demonstrating practical understanding of NLP, heuristic search, and computer vision techniques.

Project Summary

Task	Technique	Key Metric	Result
NLP Sentiment Classifier	Custom TF-IDF + Softmax	Macro F1-Score	0.843
A* Pathfinding	Manhattan Heuristic	Nodes (8×8)	39 nodes
Dijkstra	Uniform Cost Search	Nodes (8×8)	53 nodes
BFS	Level-by-Level Search	Nodes (8×8)	53 nodes
CV Pipeline	Adaptive Threshold + Contour	Avg Detections/Frame	5.6 obj
Object Tracking	Centroid Proximity	Objects Tracked	8 objects

Technologies Used

- **Python 3.14** — Core programming language
- **NumPy** — Numerical computing, matrix operations, integral image
- **Matplotlib & Seaborn** — Data visualization and charting
- **Pillow (PIL)** — Image processing, Gaussian filtering
- **ReportLab** — PDF report generation
- **Pandas & NetworkX** — Data handling and graph utilities

Key Learnings

- Implementing ML algorithms from scratch (Softmax, TF-IDF) provides deep understanding of gradient descent, loss functions, and vectorization principles.
- Heuristic search (A*) reduces search space by **29%** compared to uninformed search on 16×16 grids — demonstrating the power of domain knowledge in optimization.
- Computer vision pipelines require algorithmic optimization — integral images reduce adaptive thresholding from $O(n^3)$ to $O(n^2)$, making real-time processing feasible.
- Object tracking via centroid proximity matching provides a lightweight alternative to deep learning-based trackers for simple scenarios.

Task 1 — LinkedIn Announcement

Note: Task 1 (Professional LinkedIn Announcement) was excluded from this report due to the intern's LinkedIn account being restricted. The task will be completed once the restriction is resolved.

Project Repository & Submission

All source code, datasets, visualizations, and this report are available in the official GitHub repository for this internship.

Repository Details

Repository:<a href="https://github.com/sulmanfarooqq/Progree-internship-B1-645.git">https://github.com/sulmanf

Branch: main

Contents:tasks/, report/, README.md

Submission Checklist

- **PDF Report** — This document (18 pages, ~1.9 MB)
- **Source Code** — All Python scripts for Tasks 2, 3, 4
- **Visualizations** — 20+ PNG charts and detection images
- **README** — Project documentation with setup instructions
- **Task 1** — Pending (LinkedIn account restriction)

Note: This report was automatically generated on 10 July 2026 for **Muhammad Suliman** (ID: B1/645) as part of the Progree AI Internship. All code is original and implemented from scratch unless otherwise noted.

Progree Technologies — Remote Internship Program — 5 June 2026 – 4 July 2026
Intern: Muhammad Suliman | Student ID: B1/645
Submitted via Google Form — Ready for Evaluation